

➤ Imperative commands (single line commands to create the objects)

⌘ **kubectl run my-pod --image=nginx** [here *run* is used, not *create*]

⌘ **restartPolicy:** always

always: restart pod even if the container exits with exit code **0**

onFailure: restart pod only if container's exit code is non-zero

never: never restart the pod even if container stops.

⌘ **namespace:** default

namespace is logical grouping of a cluster to separate logically like dev, prod, teams ..etc.

⌘ **labels:** *run: my-pod*

⌘ **port:** not exposed

[port is never exposed from container though, means from Service you can route to any port of the pod you want; *port* only there for ease of querying pods according to port or to display the usable port of the container to user when they get *kubectl describe pod <pod name>*]

```
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: my-pod
    name: my-pod
spec:
  containers:
  - image: nginx
    name: my-pod
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
status: {}
```

(default configs of the command)

⌘ **kubectl create deployment my-deploy --image=nginx** [here *create* is used]

⌘ **Deployment name:** my-deploy

⌘ **ReplicaSet:** my-deploy-*<hash>*

⌘ **Pod:** my-deploy-*<hash>*-*<random>*

⌘ **labels:** *app:my-deploy*

- These are the things kubectl create commands can create. pod cannot be created using kubectl create command

clusterrole	Create a cluster role
clusterrolebinding	Create a cluster role binding for a particular cluster role
configmap	Create a config map from a local file, directory or literal value
cronjob	Create a cron job with the specified name
deployment	Create a deployment with the specified name
ingress	Create an ingress with the specified name
job	Create a job with the specified name
namespace	Create a namespace with the specified name
poddisruptionbudget	Create a pod disruption budget with the specified name
priorityclass	Create a priority class with the specified name
quota	Create a quota with the specified name
role	Create a role with single rule
rolebinding	Create a role binding for a particular role or cluster role
secret	Create a secret using a specified subcommand
service	Create a service using a specified subcommand
serviceaccount	Create a service account with the specified name
token	Request a service account token

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: my-deployment
  name: my-deployment
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-deployment
  strategy: {}
  template:
    metadata:
      labels:
        app: my-deployment
    spec:
      containers:
      - image: nginx
        name: nginx
        resources: {}
status: {}
```

(default configs of the command)

labels are automatically added.

- **ReplicaSet** cannot be created directly using imperative way, because Deployment is there to create and manage replicaset.
- **kubectl expose** takes replication controller, service, deployment, or pod and creates a *Service* to expose that.

```
vagrant@controlplane:~$ kubectl get svc,pod
NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/kubernetes                  ClusterIP           10.96.0.1     <none>         443/TCP    36m

NAME    READY    STATUS    RESTARTS    AGE
pod/my-pod  1/1     Running   0           34s
vagrant@controlplane:~$ kubectl expose pod my-pod --port 80
service/my-pod exposed
vagrant@controlplane:~$ kubectl get svc,pod
NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/kubernetes                  ClusterIP           10.96.0.1     <none>         443/TCP    37m
service/my-pod                      ClusterIP           10.96.68.232  <none>         80/TCP     6s

NAME    READY    STATUS    RESTARTS    AGE
pod/my-pod  1/1     Running   0           76s
```

you can see here, It created a service named *my-pod*. Its of type **ClusterIP** which is default.

- If you give **--type=NodePort** flag, then it'll create a service of type NodePort

```
vagrant@controlplane:~$ kubectl expose pod my-pod --port 80 --type NodePort
service/my-pod exposed
vagrant@controlplane:~$ kubectl get svc,pod
NAME                                TYPE                CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
service/kubernetes                  ClusterIP           10.96.0.1     <none>         443/TCP    39m
service/my-pod                      NodePort            10.96.119.232 <none>         80:32530/TCP 3s
```

The collage shows several terminal windows with the following commands:

- `$ kubectl run --image=nginx nginx`
- `$ kubectl expose deployment nginx --port 80`
- `$ kubectl scale deployment nginx --replicas=5`
- `$ kubectl create -f nginx.yaml`
- `$ kubectl delete -f nginx.yaml`
- `$ kubectl create deployment --image=nginx nginx`
- `$ kubectl edit deployment nginx`
- `$ kubectl set image deployment nginx nginx=nginx:1.18`
- `$ kubectl replace -f nginx.yaml`

[these are the imperative commands]

create -f is imperative because if already the same config present it'll fail, where as **apply -f** will not fail.

➤ Important trick

- **kubectl create deployment my-deploy --image=nginx --dry-run=client -o yaml**

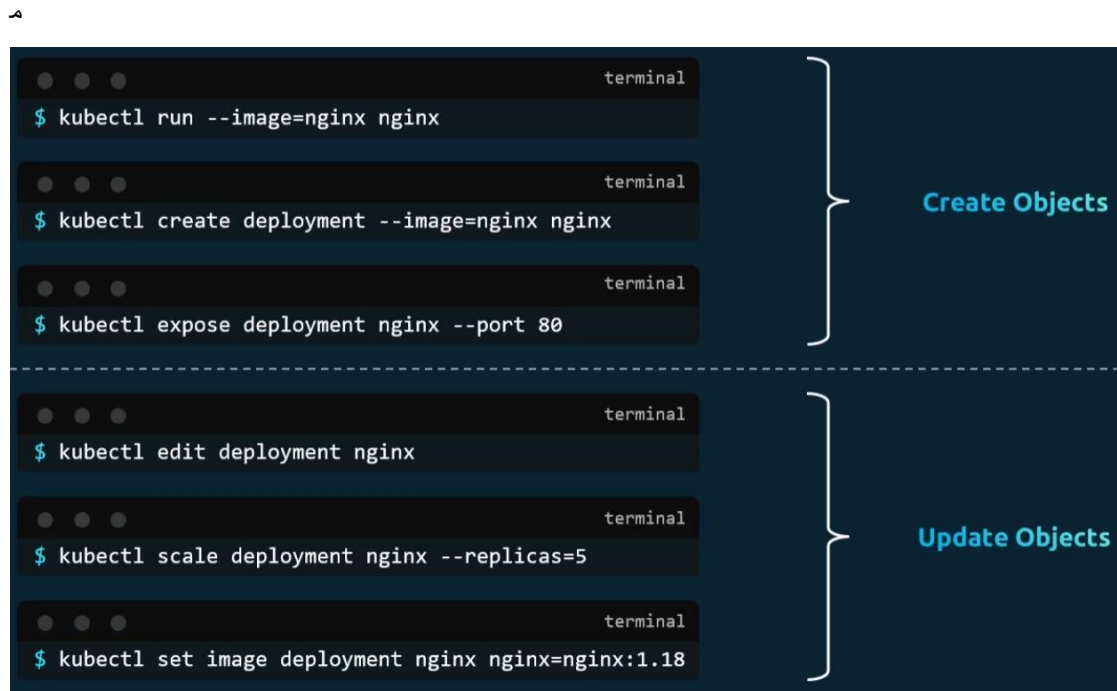
kubectl run my-pod --image=nginx --dry-run=client -o yaml

- it'll give the yaml file of default deployment, remove the unnecessary fields, customize that, and create deployment

- Because of the flag **--dry-run=client** it'll not create the deployment, just simulate and give the output as yaml.

➤ **kubectl get pod my-pod -o yaml**

- It'll give the running config of the pod as an yaml.
- Can be done with any types of objects i.e. pod, replica set, container runtime, deployment ..etc
- But it'll contain a lot of things, that needs to be removed.



➤ **edit vs replace command**

- kubectl edit** command open the running status of the object (pod, replicaset, replication controller, deployment, service ..etc) in a yaml file.

This configurations are there in the memory, not stored in the yaml file that was being used to create the object.

So, if you do **kubectl edit** then it'll not have any track of the changes. If someone later want to update the object, and he sees the yaml file present (not the running configuration yaml) and try to update, it might do some wrong changes in the running object.

- kubectl replace** command uses the physical yaml file to update the running object.

It does **dumb** update in the running object. Whatever there in the physical yaml, it'll update all the things and delete the configs which are not present in that physical yaml file.

By default it doesn't delete the current object (if anything doesn't match

with physical yaml), to replace force fully you can give the flag **--force**.

```
vagrant@controlplane:~$ kubectl replace --force -f pd.yaml
pod "my-pod" deleted from default namespace
pod/my-pod replaced
```

It looks similar to *kubectl apply*, but *kubectl apply* does **intelligent** update. It finds the **diff** between running object and the physical yaml (used to update the object) and update those only.

It **doesn't delete any configs** which are not present in the current yaml but there in the running object.

kubectl apply is **declarative** command.

- If there are multiple files out of which objects will have to be created, then just give the path of the directory where those are present to *kubectl apply*, it'll create all those.

kubectl apply -f <path where all files are present>

--- kubectl apply internals ---

➤ [here I'll take an example of a deployment named *my-dep*]

➤ There are 3 things.

yaml (file that you write)

last applied config (when you execute *kubectl get deploy my-dep -o yaml*) [JSON format]

live config (when you execute *kubectl describe deploy my-dep*)

[*deploy* is shorthand of *deployment*]

⌘ Whenever you do *declarative* change using a *yaml* i.e.

kubectl apply -f my-dep.yaml

it changes both *live config*, *last applied config*. and obviously *yaml* is changed by you only.

⌘ Whenever you do *imperative* change using the command i.e.

kubectl scale deployment my-dep --replicas=5

it changes only *live config*. *Last applied config* will not be changed. And of course, *yaml* has not been changed.

command type	yaml	last applied config	live config
declarative	Y	Y	Y
imperative	N	N	Y

➤ Lets see the below example: -----

➤ Stage-1; *declarative*; using *kubectl apply -f my-dep.yaml*

⌘ The below is my initial *yaml* config

```
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: my-dep
  name: my-dep
spec:
  replicas: 2
  selector:
    matchLabels:
      app: my-dep-pod
  strategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: my-dep-pod
    spec:
      containers:
        - image: nginx:1.29
          name: my-dep-cont
          ports:
            - containerPort: 80
              name: nginx-port
```

[just look into *replicas*, *image*]

⌘ replicas: **2** ; image: **nginx:1.29**

⌘ **Last applied configuration** [**kubectl get deploy my-dep -o yaml**] :

```
{"apiVersion":"apps/v1","kind":"Deployment","metadata":{"annotations":{"label
s":{"app":"my-dep"},"name":"my-
dep","namespace":"default"},"spec":{"replicas":2,"selector":{"matchLabels":{"app":
"my-dep-
pod"}},"strategy":{"type":"RollingUpdate"},"template":{"metadata":{"labels":{"ap
p":"my-dep-pod"}},"spec":{"containers":[{"image":"nginx:1.29","name":"my-dep-
cont"},"ports":[{"containerPort":80,"name":"nginx-port"}]}}}}}
```

```
vagrant@controlplane:~$ kubectl get deploy my-dep -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  annotations:
    deployment.kubernetes.io/revision: "1"
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"Deployment","metadata":
      {"name":"my-dep","namespace":"default"},"spec":{"replicas":2,"selector":{"matchLabels":{"app":"my-dep-pod"}},"strategy":{"type":"RollingUpdate"},"template":{"metadata":{"labels":{"app":"my-dep-pod"}},"spec":{"containers":[{"image":"nginx:1.29","name":"my-dep-cont"},"ports":[{"containerPort":80,"name":"nginx-port"}]}}}}
  creationTimestamp: "2026-04-30T19:58:39Z"
```

[template.spec.containers.image: **nginx:1.29**]

[the last-applied-configuration stays inside annotation field]

```
spec:
  progressDeadlineSeconds: 600
  replicas: 2
```

[spec.replicas: 2]

⌘ **Live configuration** [**kubectl edit deploy my-dep** (or) **kubectl describe deploy my-dep**]

[even in *kubectl edit* command you'll see that json (last-applied-config), **don't change this**. If you do then it'll update the last-applied-config which is dangerous.

Will discuss this]

```
spec:
  progressDeadlineSeconds: 600
  replicas: 2
```

[spec.replicas: 2]

```
spec:
  containers:
  - image: nginx:1.29
    imagePullPolicy: IfNotPresent
    name: my-dep-cont
```

[template.spec.containers.image: **nginx:1.29**]

- Stage-2; **imperative**; using *kubectl set image deploy my-dep my-dep-cont=nginx:1.30*

[only image will be changed here keeping the replicas same]

- ⌘ *yaml* has not changed. So

replicas: **2** ; image: *nginx:1.29* [in yaml]

- ⌘ *last-applied-config*

```
{
  "containers": [
    {
      "image": "nginx:1.29",
      "name": "my-dep-cont",
      "ports": [
        {
          "containerPort": 80,
          "name": "nginx-port"
        }
      ]
    }
  ],
  "spec": {
    "replicas": 2,
    "selector": {
      "matchLabels": {
        "app": "my-dep-pod"
      }
    },
    "strategy": {
      "type": "RollingUpdate"
    },
    "template": {
      "metadata": {
        "labels": {
          "app": "my-dep-pod"
        }
      },
      "spec": {
        "containers": [
          {
            "image": "nginx:1.29",
            "name": "my-dep-cont",
            "ports": [
              {
                "containerPort": 80,
                "name": "nginx-port"
              }
            ]
          }
        ]
      }
    }
  }
}
```

replicas: **2** ; image: *nginx:1.29* [in last-applied-config]

it didn't change because the command executed was of *imperative*.

- ⌘ *live-config*

```
spec:
  progressDeadlineSeconds: 600
  replicas: 2
```

[because we have not changed]

```
spec:
  containers:
  - image: nginx:1.30
    imagePullPolicy: IfNotPresent
    name: my-dep-cont
```

[image changed to *nginx:1.30*]

replicas: **2** ; image: *nginx:1.30* [in live-config]

- Stage-3; **declarative**; using *kubectl apply -f my-dep.yaml*

[only replicas will be changed here]

```
spec:
  replicas: 4
  selector:
    matchLabels:
      app: my-dep-pod
  strategy:
    type: RollingUpdate
  template:
    metadata:
      labels:
        app: my-dep-pod
    spec:
      containers:
      - image: nginx:1.29
        name: my-dep-cont
        ports:
        - containerPort: 80
          name: nginx-port
```

- ⌘ [replicas: 4; image is showing 1.29 because we have not changed this yaml; stage-2 change was imperative]

- ⌘ *yaml*

replicas: **4** ; image: *nginx:1.29* [in yaml]

^ *last-applied-config*

```
"spec":{"containers":[{"image":"nginx:1.29"},  
"spec":{"replicas":4,
```

replicas: **4**; image: **nginx:1.29** [in last-applied-config]

replicas got updated because the change was *declarative*.

^ *live-config*

```
spec:  
  progressDeadlineSeconds: 600  
  replicas: 4
```

```
spec:  
  containers:  
    - image: nginx:1.29
```

replicas: **4**; image: **nginx:1.29** [in last-applied-config]

➤ F

➤ F

➤ F

➤ F

➤ kubectl diff -f simple_deployment.yaml

kubectl get -f simple_deployment.yaml -o yaml

kubectl apply view-last-updated

➤ F

➤ F

➤ F

➤ Replicas nginx

1 1.14.2 (apply)

➤ kubectl apply

Sets fields, that appear in the configuration file, in the live configuration.

Clears fields, that removed from the configuration file, in the live configuration.

➤ F

➤ F

➤ F

➤ F

➤ F

➤ F

- **F**
- **F**
- **F**
- **F**
- **f**
- **F**

Important Points

- Try to use *imperative commands* as much as you can in Exam.
- If you forget the keywords and resources, then execute **kubect! api-resources** you'll get the keywords.

```
vagrant@controlplane:~$ kubectl api-resources
```

NAME	SHORTNAMES	APIVERSION	NAMESPACED	KIND
bindings		v1	true	Binding
componentstatuses	cs	v1	false	ComponentStatus
configmaps	cm	v1	true	ConfigMap
endpoints	ep	v1	true	Endpoints
events	ev	v1	true	Event
limitranges	limits	v1	true	LimitRange
namespaces	ns	v1	false	Namespace
nodes	no	v1	false	Node
persistentvolumeclaims	pvc	v1	true	PersistentVolumeClaim
persistentvolumes	pv	v1	false	PersistentVolume
pods	po	v1	true	Pod
podtemplates		v1	true	PodTemplate
replicationcontrollers	rc	v1	true	ReplicationController
resourcequotas	quota	v1	true	ResourceQuota
secrets		v1	true	Secret
serviceaccounts	sa	v1	true	ServiceAccount
services	svc	v1	true	Service
mutatingadmissionpolicies		admissionregistration.k8s.io/v1	false	MutatingAdmissionPolicy
mutatingadmissionpolicybindings		admissionregistration.k8s.io/v1	false	MutatingAdmissionPolicyBinding
mutatingwebhookconfigurations		admissionregistration.k8s.io/v1	false	MutatingWebhookConfiguration

[you can see there is a column 'KIND' which you can directly write in the yaml]

- ⌘ **Shortnames** are also there to use in *imperative commands* which will SAVE TIME.
- If you want to get a documentation regarding any specific resource, you can use **kubect! explain <resource name>** command.

```
vagrant@controlplane:~$ kubectl explain pods
```

```
KIND:      Pod
VERSION:   v1
```

DESCRIPTION:

Pod is a collection of containers that can run on a host. This resource is created by clients and scheduled onto hosts.

FIELDS:

```
apiVersion  <string>
  APIVersion defines the versioned schema of this representation of an object.
  Servers should convert recognized schemas to the latest internal value, and
  may reject unrecognized values. More info:
  https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#resources

kind  <string>
  Kind is a string value representing the REST resource this object
  represents. Servers may infer this from the endpoint the client submits
  requests to. Cannot be updated. In CamelCase. More info:
  https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#types-kinds

metadata    <ObjectMeta>
  Standard object's metadata. More info:
  https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#metadata

spec  <PodSpec>
  Specification of the desired behavior of the pod. More info:
  https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status

status    <PodStatus>
  Most recently observed status of the pod. This data may not be up to date.
  Populated by the system. Read-only. More info:
  https://git.k8s.io/community/contributors/devel/sig-architecture/api-conventions.md#spec-and-status
```

- To go deeper into the specific fields of that resources,

kubectl explain <resource name>.<field name>

```
vagrant@controlplane:~$ kubectl explain pods.spec
KIND:      Pod
VERSION:   v1

FIELD: spec <PodSpec>

DESCRIPTION:
  Specification of the desired behavior of the pod. More info:
  https://git.k8s.io/community/contributors/devel/sig-architecture/api-convent
  PodSpec is a description of a pod.

FIELDS:
  activeDeadlineSeconds <integer>
    Optional duration in seconds the pod may be active on the node relative to
```

If you want, more comprehensive, then use

kubectl explain <resource name> --recursive

```
vagrant@controlplane:~$ kubectl explain pods --recursive
KIND:      Pod
VERSION:   v1

DESCRIPTION:
  Pod is a collection of containers that can run on a hos
  created by clients and scheduled onto hosts.

FIELDS:
  apiVersion    <string>
  kind          <string>
  metadata      <ObjectMeta>
    annotations <map[string]string>
    creationTimestamp <string>
    deletionGracePeriodSeconds <integer>
```

- To get a basic yaml file instead of writing from scratch, use ***--dry-run=client -o yaml*** with the command.

[with *run*, *create* for static] [with *get* if the object is already running]

kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml

kubectl run my-pod --image nginx --dry-run client -o yaml

```
vagrant@controlplane:~$ kubectl run my-pod --image=nginx --dry-run=client -o yaml
apiVersion: v1
kind: Pod
metadata:
  labels:
    run: my-pod
  name: my-pod
spec:
  containers:
  - image: nginx
    name: my-pod
    resources: {}
  dnsPolicy: ClusterFirst
  restartPolicy: Always
```

```
vagrant@controlplane:~$ kubectl create deployment my-dep --image=nginx --dry-run=client -o yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  labels:
    app: my-dep
  name: my-dep
spec:
  replicas: 1
  selector:
    matchLabels:
      app: my-dep
  strategy: {}
  template:
    metadata:
      labels:
        app: my-dep
    spec:
      containers:
      - image: nginx
        name: nginx
        resources: {}
```

➤

➤ F

➤ F

➤ F

➤ F

➤ F

➤ F

➤